

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

FACULTAD DE CIENCIAS DE LA COMPUTACIÓN – INGENIERÍA DE SOFTWARE

Asignatura: Aplicaciones Distribuidas (ISR-701) | **Docente Revisor:** Prof. Ing. Gleiston C. Guerrero-Ulloa, Mgs.

GUÍA MAESTRA DE DEFENSA ORAL INDIVIDUAL – PROYECTO FIN DE CURSO (PFC E4)

AcadTrace: Gestión Académica Distribuida con Auditoría Criptográfica Verificable

Estudiante Defensor:	Ernesto Gregory Luna Mora (Ernesto835 <elunam4@uteq.edu.ec>)	Equipo: BCEL
Rol Oficial Asignado:	Responsable de Microservicio Secretaría y Calidad	Criterio: C10 (Defensa Oral)

1. Estrategia de Presentación y Discurso de Apertura (2 Minutos)

Cuando el docente inicie la evaluación individual y solicite: “Luna, presénteme su parte y qué aportó al proyecto”, debes intervenir con total aplomo técnico usando este guión estructurado:

*“Buenos días Ingeniero Guerrero. En el equipo BCEL mi rol formal ha sido **Responsable del Microservicio de Secretaría y Aseguramiento de la Calidad**.*

*Mi responsabilidad se concentró en resolver el desafío nuclear que le da nombre a AcadTrace: **¿cómo garantizar no-repudio e inmutabilidad estricta de notas, incluso si un atacante obtiene credenciales de superusuario en la base de datos?***

*Para responder a esto, implementé una arquitectura de **defensa en profundidad** articulada en 4 componentes:*

1. **A nivel de base de datos:** Un disparador PostgreSQL **Append-Only** que intercepta físicamente cualquier sentencia UPDATE o DELETE.
2. **A nivel de servicio:** Sellado criptográfico mediante **HMAC-SHA256 con encadenamiento sucesivo (H_i)** y cifrado simétrico autenticado **AES-256-GCM** para blindar datos de menores de edad (Criterio Ético C9).
3. **A nivel de calidad (C7 y Listado 3):** Construí e integré la **suite de 21 pruebas automatizadas** de contratos Protobuf v3, integración de red y ciclo E2E.
4. **A nivel empírico (C2, C3, P4):** Certifiqué experimentalmente una tasa de falsos positivos **FPR = 0.00%** en 30 cadenas legítimas con IC al 95 % de [0,0 %, 11,6 %], respaldado con certificados SHA-256.

Tengo en pantalla el código fuente y las pruebas en vivo para sustentar cada evidencia.”

2. Los Cuatro Pilares Técnicos que Debes Defender

Pilar 1: Microservicio Secretaría e Inmutabilidad en Persistencia (C1, Tema 10)

Emite actas y matrículas. A nivel de PostgreSQL implementamos el trigger `tg_auditoria_append_only` que intercepta sentencias BEFORE UPDATE OR DELETE en `sga_principal.auditoria` y ejecuta `prohibir_modificacion_aud` lanzando un RAISE EXCEPTION. Esto asegura persistencia inmutable en disco.

Pilar 2: Auditoría Criptográfica y Protección de Menores (C1, C9, Temas 9 y 10)

El sellado se calcula como $H_i = \text{HMAC-SHA256}(K, \text{payload}_i \parallel H_{i-1})$. En `HmacService.java` la verificación se realiza mediante `MessageDigest.isEqual` en **tiempo constante ($O(1)$)** para neutralizar ataques de temporización (*Timing Attacks*). Los datos de contacto y cédulas se cifran con **AES-256-GCM** con IV aleatorio de 12 bytes y tag de 128 bits (`CryptoService.java`), acatando la LOPDP ecuatoriana.

Pilar 3: Suite Automatizada de Calidad (Criterio C7 y Listado 3)

Respondiendo a la observación del informe provisional, construimos 21 pruebas en Python:

- **Contratos (tests/contract/ - 7 tests):** Sintaxis proto3, paridad de métodos RPC y OpenAPI 3.0 con JWT.
- **Integración (tests/integration/ - 8 tests):** HAProxy roundrobin/leastconn, multi-esquema y X-Trace-Id.
- **Ciclo E2E (tests/e2e/ - 6 tests):** Rechazo 401, JWT HMAC, detección T1/T2 y reconciliación vectorial M3.

Pilar 4: Sinceramiento Empírico y Falsos Positivos (C2, C3, P4)

Sustituimos métricas cableadas por extracción real de Locust (106,735 peticiones de estrés) y JaCoCo (100 % Cripto-Core, 34.52 % Secretaría). En `falsos_positivos.csv` se validaron 30 cadenas legítimas sin adulterar obteniendo $FPR = \frac{0}{30} = 0,00 \%$.

3. Catálogo de Código Fuente: Ubicación Exacta, Snippets y Función

Ten abiertas estas 6 pestañas en tu editor de código durante la reunión:

Pieza 1: Disparador SQL Append-Only en PostgreSQL

Ubicación: sga-principal/sql/V9_trigger_auditoria_append_only.sql (Líneas 11–26) y docs/db/schema.sql

Función: Aborta físicamente cualquier operación destructiva (UPDATE/DELETE) sobre la bitácora.

```

1 CREATE OR REPLACE FUNCTION sga_principal.prohibir_modificacion_auditoria()
2 RETURNS TRIGGER AS $$
3 BEGIN
4     RAISE EXCEPTION 'Operacion rechazada: La tabla sga_principal.auditoria es una bitacora inmutable de solo
5     adicion (append-only) protegida bajo estandares ISO/IEC 25010';
6 END;
7 $$ LANGUAGE plpgsql;
8
9 CREATE TRIGGER tg_auditoria_append_only
10 BEFORE UPDATE OR DELETE ON sga_principal.auditoria
11 FOR EACH ROW EXECUTE FUNCTION sga_principal.prohibir_modificacion_auditoria();

```

Pieza 2: Firma HMAC-SHA256 Timing-Safe en Secretaría

Ubicación: microservicio-secretaria/backend/.../infrastructure/security/HmacService.java (Líneas 33–54)

Función: Computa la firma de 64 caracteres hex y compara en tiempo constante para evitar fugas de canal lateral.

```

1 public synchronized String firmar(String... campos) {
2     String canonical = String.join("|", campos);
3     byte[] digest = mac.doFinal(canonical.getBytes(StandardCharsets.UTF_8));
4     StringBuilder hex = new StringBuilder(digest.length * 2);
5     for (byte b : digest) hex.append(String.format("%02x", b));
6     return hex.toString();
7 }
8
9 public boolean verificar(String hmacEsperado, String... campos) {
10     if (hmacEsperado == null || hmacEsperado.isBlank()) return false;
11     String calculado = firmar(campos);
12     // Comparacion en tiempo constante O(1) para prevenir Timing Attacks
13     return java.security.MessageDigest.isEqual(
14         hmacEsperado.getBytes(StandardCharsets.UTF_8),
15         calculado.getBytes(StandardCharsets.UTF_8)
16     );
17 }

```

Pieza 3: Cifrado Simétrico Autenticado AES-256-GCM para Menores

Ubicación: microservicio-secretaria/backend/.../infrastructure/security/CryptoService.java (Líneas 25–61)

Función: Cifra datos de menores con IV aleatorio de 12 bytes y tag GCM de 128 bits, sin almacenar el IV en texto plano.

```

1 private static final String ALGORITHM = "AES/GCM/NoPadding";
2 private static final int GCM_IV_LENGTH = 12;
3 private static final int GCM_TAG_LENGTH_BITS = 128;
4
5 public String encrypt(String plaintext) {
6     if (plaintext == null) return null;
7     try {
8         byte[] iv = new byte[GCM_IV_LENGTH];
9         random.nextBytes(iv);
10        Cipher cipher = Cipher.getInstance(ALGORITHM);
11        cipher.init(Cipher.ENCRYPT_MODE, key, new GCMParameterSpec(GCM_TAG_LENGTH_BITS, iv));
12        byte[] ciphertext = cipher.doFinal(plaintext.getBytes(StandardCharsets.UTF_8));
13
14        byte[] output = new byte[iv.length + ciphertext.length];
15        System.arraycopy(iv, 0, output, 0, iv.length);
16        System.arraycopy(ciphertext, 0, output, iv.length, ciphertext.length);
17        return Base64.getEncoder().encodeToString(output);
18    } catch (GeneralSecurityException e) {
19        throw new IllegalStateException("Error cifrando dato sensible", e);
20    }
21 }

```

Pieza 4: Prueba Unitaria JUnit 5 de Detección de Manipulación T1

Ubicación: microservicio-secretaria/backend/.../security/AuditoriaIntegridadTest.java (Líneas 62–81)

Función: Demuestra que al alterar el `id_registro` en BD (Ataque T1), el validador rechaza la firma.

```

1 @Test
2 @DisplayName("Debe detectar inmediatamente manipulacion directa en id_registro (Ataque T1)")
3 void testDeteccionManipulacion_RegistroIdAlterado() {
4     String firmaOriginal = hmacService.firmar(schema, traceId, username, accion, tabla,
5                                             registroIdOriginal, descripcion, resultado, timestamp);
6
7     // Al contrastar frente al registro adulterado (999), DEBE ser rechazada
8     boolean esValido = hmacService.verificar(firmaOriginal, schema, traceId, username, accion, tabla,
9                                             registroIdAdulterado, descripcion, resultado, timestamp);
10    assertFalse(esValido, "El sistema debe detectar la manipulacion del id de registro");
11 }

```

Pieza 5: Detección de Borrado T2 y Reconciliación Vectorial M3 (E2E)

Ubicación: tests/e2e/test_e2e_lifecycle.py (Líneas 146–172)

Función: Verifica ruptura de hash al mutilar eventos intermedios y resolución de concurrencia offline ($A \parallel B$).

```

1 # Deteccion de Borrado T2: Se mutila el evento intermedio
2 eventos_mutilados = [eventos[0], eventos[2]]
3 valida, rule_desc, broken_id, elapsed_us = verificar_cadena_eventos(eventos_mutilados, "M2")
4 assert not valida, "El borrado de eventos intermedios (T2) debe romper la cadena SHA-256"
5
6 # Deteccion de Concurrencia Causal en Relojes Vectoriales: A || B
7 es_concurrente = not (all(x >= y for x, y in zip(v_docA, v_docB)) or all(x <= y for x, y in zip(v_docA, v_docB)))
8 assert es_concurrente, "Debe identificarse conflicto causal concurrente (A || B)"
9
10 # Reconciliacion determinista M3: merge componente a componente
11 v_reconciliado = [max(x, y) for x, y in zip(v_docA, v_docB)]
12 nota_reconciliada = max(nota_A, nota_B)

```

Pieza 6: Certificación de Cero Falsos Positivos (falsos_positivos.csv)

Ubicación: experimentos/resultados/falsos_positivos.csv y docs/TRAZABILIDAD.md

Función: Evidencia empírica de 30 corridas sin alterar con 50 eventos cada una:

$$\text{FPR} = \frac{\text{FP}}{\text{FP} + \text{TN}} = \frac{0}{0 + 30} = 0,00\% \quad [\text{IC } 95\% \text{ Clopper-Pearson: } [0,0\%, 11,6\%], \quad \bar{t}_{\text{verif}} = 0,15 \text{ ms}]$$

4. Banco de Preguntas Trampa del Docente y Respuestas Modelo

1. ¿Por qué una base de datos con triggers no basta para garantizar no-repudio? *Respuesta:* El trigger es una defensa en profundidad en runtime, pero un atacante interno con rol `postgres` (DBA) puede ejecutar `ALTER TABLE ... DISABLE TRIGGER` o manipular bloques en disco. Por eso AcadTrace incorpora una capa criptográfica matemática independiente (H_i): si el atacante elude el trigger, la verificación de hashes romperá la cadena inmediatamente.
2. ¿Cuál es la diferencia entre el Reloj de Lamport y los Relojes Vectoriales en el proyecto? *Respuesta:* El Reloj de Lamport ($L = \max(L_{\text{loc}}, L_{\text{rx}}) + 1$) genera un orden total causal débil ($A \rightarrow B \implies L(A) < L(B)$), pero no al revés; lo usamos en Secretaría para secuenciar transacciones. El Reloj Vectorial ($\vec{V}$) mantiene un reloj por actor y detecta concurrencia real ($A \parallel B$); lo usamos en Docente para reconciliar notas editadas offline en la app móvil.
3. En la matriz ISO 25010 la cobertura de SGA Principal es 0.51 % y la de Secretaría es 34.52 %. ¿Por qué? *Respuesta:* Por sinceramiento empírico (Criterio C3). Antes se presentaban valores teóricos del 85 %. Al auditar JaCoCo real, SGA Principal concentra entidades JPA y código autogenerado por `protoc`. En Secretaría alcanzamos el 34.52 % global y el 100 % en la lógica de negocio criptográfica (`AuditoriaIntegridadTest`). Además, las 21 pruebas E2E en Python cubren los flujos inter-servicios de caja negra.
4. ¿Dónde se evidencia la revisión cruzada y los issues de GitHub? (Criterio C8) *Respuesta:* En la Sección 8 del informe LaTeX y en el repositorio. Vinculamos cada issue con su PR y revisores: #38 y #41 (Auditoría append-only), #44 (JaCoCo), #47 (gRPC Principal-Docente), #50 y #53 (Locust y Grafana), y #57 (Certificado C4), más co-evaluación con rúbrica ciega.

5. Hoja de Comandos Rápidos para Demostración en Vivo (PowerShell)

En la carpeta `C:\Users\DEYNER\acadtrace`:

1. **Correr las 21 pruebas automatizadas (C7):** `python -m pytest tests/contract tests/integration tests/e2e -v`
2. **Correr experimentos y cálculo de FPR:** `python experimentos/run_experimentos.py`
3. **Verificar certificado de reproducibilidad C4:** `cat experimentos/resultados/REPRODUCIBILIDAD.txt`
4. **Mostrar tu Pull Request #60 fusionado en main:** `git log -n 3 -pretty=format:"%h |%an |%s"`